

LAB EXPERIMENT REPORT

Object-Oriented Programming using Java

Basic Information	
Experiment No.	_____
Date	_____
Title	Roman Numeral to Integer Converter in Java

1. Objective / Aim

To design and implement a Java program that converts a Roman numeral (e.g., MCMXCIV) entered by the user into its equivalent integer value (1994), using methods, a switch statement, loops and the subtractive-notation rule of Roman numbers.

2. System Requirements

Hardware Requirements

- Processor: Intel Core i3 / AMD equivalent or higher
- RAM: 4 GB minimum (8 GB recommended)
- Hard disk: 500 MB free space
- Input/Output: Keyboard, monitor

Software Requirements

- Operating System: Windows 10/11, Linux or macOS
- Java Development Kit (JDK) 8 or higher
- Editor / IDE: Eclipse, IntelliJ IDEA, NetBeans or VS Code
- Command Prompt / Terminal for compilation and execution

3. Algorithm / Logic Description

Roman symbol values:

Symbol	I	V	X	L	C	D	M
Value	1	5	10	50	100	500	1000

Key idea: Roman numerals are normally written largest to smallest and values are added. If a smaller symbol appears *before* a larger one (IV, IX, XL, XC, CD, CM), its value is *subtracted* instead.

Algorithm:

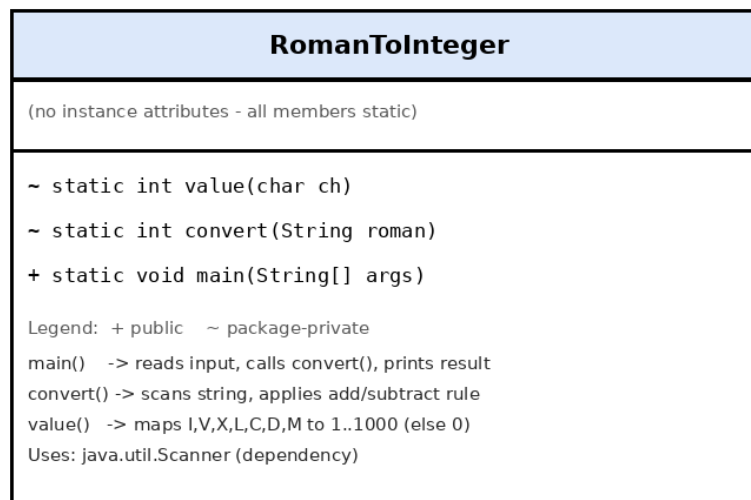
1. Start.
2. Read the Roman number as a string and convert it to upper case.
3. Set result = 0.
4. For each character at position i, find its value (current) using value().

5. If a next character exists and current < value of next character, then result = result - current.
6. Otherwise, result = result + current.
7. Repeat steps 4-6 until the end of the string.
8. Display result.
9. Stop.

Worked example (MCMXCIV): M(+1000) C(-100, since C<M) M(+1000) X(-10, since X<C) C(+100, last C is followed by smaller I) I(-1, since I<V) V(+5) gives 1994.

Complexity: Time $O(n)$, Space $O(1)$, where n is the length of the input string.

4. UML Class Diagram



Package: roman

5. Source Code

```
package roman;

import java.util.Scanner;

public class RomanToInteger {

    static int value(char ch) {
        switch (ch) {
            case 'I': return 1;
            case 'V': return 5;
            case 'X': return 10;
            case 'L': return 50;
            case 'C': return 100;
            case 'D': return 500;
            case 'M': return 1000;
            default: return 0;
        }
    }

    static int convert(String roman) {
        int result = 0;

        for (int i = 0; i < roman.length(); i++) {
            int current = value(roman.charAt(i));

            if (i + 1 < roman.length() &&
                current < value(roman.charAt(i + 1))) {
                result -= current;
            } else {
                result += current;
            }
        }

        return result;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Roman Number: ");
        String roman = sc.nextLine().toUpperCase();

        int result = convert(roman);

        System.out.println("Integer Value: " + result);

        sc.close();
    }
}
```

6. Compilation and Execution Commands

The file is saved as **RomanToInteger.java** inside a folder named **roman** (because of the line `package roman;`). Open a terminal in the parent folder of *roman* and run:

```
javac roman/RomanToInteger.java
java roman.RomanToInteger
```

Alternative (without a package folder): remove the `package roman;` line, then run `javac RomanToInteger.java` and `java RomanToInteger`.

7. Output

```
Terminal - Roman to Integer

$ cd ~/project
$ javac roman/RomanToInteger.java
$ java roman.RomanToInteger
Enter Roman Number: MCMXCIV
Integer Value: 1994

$ java roman.RomanToInteger
Enter Roman Number: xlii
Integer Value: 42

$ java roman.RomanToInteger
Enter Roman Number: MMXXVI
Integer Value: 2026
$ _
```

Input	Expected	Actual Output
MCMXCIV	1994	1994
xlii	42	42
MMXXVI	2026	2026

8. Result / Conclusion

The Java program successfully converts Roman numerals to integers by scanning the string once and applying the add/subtract rule. Lower-case input is handled by `toUpperCase()`. Note: the program assumes a valid Roman numeral; invalid characters are treated as value 0.